

Recorregut de Dues Peces en un Tauler

Pràctica 2

Serafí Nebot Ginard
Universitat de les Illes Balears (UIB)
Grau en Enginyeria Informàtica
Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria descriu el disseny i la implementació d'una aplicació Java que resol el *problema del recorregut de dues peces*: donat un tauler $N \times M$ i dues peces d'escacs, determinar si existeix un camí hamiltonià en el qual les dues peces s'alternen per cobrir totes les caselles exactament una vegada, sense que cap de les dues pugui capturar l'altra en cap moment. L'aplicació implementa el patró MVC amb comunicació basada en events semàntics, utilitza backtracking recursiu millorat amb l'heurística de Warnsdorff, i incorpora una interfície gràfica interactiva que permet configurar el tauler, visualitzar el procés de resolució en temps real i reproduir la solució pas a pas.

Termes d'índex—backtracking, heurística de Warnsdorff, camí hamiltonià, Model-Vista-Controlador, SwingWorker, peces d'escacs, Java Swing

1. Introducció

El problema del recorregut del cavall (*knight's tour*) és un problema clàssic de la teoria de grafs i dels algorismes combinatoris: donat un tauler d'escacs, cal trobar una seqüència de moviments d'un cavall tal que visiti cada casella exactament una vegada. La generalització d'aquest problema a dues peces que s'alternen els moviments, amb la restricció addicional que cap de les dues pot capturar l'altra en cap pas del recorregut, és notablement més complexa.

L'aplicació implementada permet seleccionar dos tipus de peces de manera independent (Cavall, Reina, Torre, Alfíl, Mussol o Serp) i trobar, mitjançant backtracking, un recorregut alternat que cobreixi totes les caselles d'un tauler de mida configurable (mínim 2×2 , sense límit superior pràctic).

2. Descripció de l'Aplicació

L'aplicació és una eina interactiva d'escriptori (Java Swing) que permet plantejar i resoldre el problema del recorregut hamiltonià de dues peces, visualitzar la solució pas a pas i inspeccionar les mètriques de la cerca. La interfície es divideix en tres zones: un panell de configuració a l'esquerra, el tauler al centre i un panell de controls i estadístiques a la part inferior.

2.1. Configuració

2.1.1. Mida del tauler. L'usuari especifica el nombre de files i columnes amb dos *spinners* (mínim 2, sense límit superior pràctic). Un botó *toggle* etiquetat *Quadrat*, situat sobre els dos *spinners* i actiu per defecte, manté les dues

dimensions iguals: qualsevol canvi en un *spinner* es propaga automàticament a l'altre. Quan el botó està inactiu, files i columnes es poden ajustar de manera independent per obtenir taulers no quadrats ($N \times M$). El botó canvia de color de fons (verd quan actiu) per indicar l'estat d'un cop d'ull.

2.1.2. Selecció de peces. Cada una de les dues peces (blanca i negra) es configura de manera independent mitjançant un `JComboBox` amb les sis opcions disponibles: Cavall, Reina, Torre, Alfíl, Mussol i Serp. Una icona en miniatura mostra la imatge de la peça seleccionada (o les inicials en un cercle si la imatge no està disponible).

2.1.3. Posició inicial. La posició inicial de cada peça es pot establir de dues maneres:

- Introduint directament els valors de fila i columna (en base 1) als *spinners* de posició.
- Activant el mode de col·locació amb el botó *Col·locar al tauler* (color de fons taronja quan actiu) i fent clic sobre la casella desitjada al tauler. El mode es desactiva automàticament després de col·locar la peça.

En tot moment, les dues peces es mostren sobre el tauler buit en la seva posició configurada com a previsualització.

2.2. Execució de la cerca

El panell de controls inferior ofereix tres botons d'execució:

- **Iniciar:** valida la configuració, construeix el tauler i llança la cerca en un fil de fons.
- **Aturar:** envia un senyal cooperatiu al solver per a que aturi la cerca en curs.
- **Reiniciar:** atura la cerca (si n'hi ha una en curs), buida el tauler i restaura l'estat inicial.

Durant la cerca, tots els controls de configuració es deshabiliten per evitar inconsistències. El panell d'estadístiques mostra en temps real el nombre d'iteracions, de retrocesos, la profunditat màxima assolida i el temps transcorregut. Un indicador d'estat codificat per colors informa sobre la fase actual: gris (*Esperant*), blau (*Cercant*), verd (*Solució trobada*), vermell (*No hi ha solució*) i groc-marró (*Aturat*). Els colors específics es detallen a la Taula 2.

2.3. Visualització de la solució

Si la cerca troba una solució, el tauler es mostra complet amb totes les caselles visitades. Cada casella visitada indica el número de pas a la cantonada superior esquerra, i les icones de cada peça es mostren a la seva posició final. El tauler inclou etiquetes de fila (a l'esquerra) i columna (a la part superior), que s'alineen exactament amb cada casella.

L'usuari pot navegar per la solució pas a pas:

- **<< / >>:** retrocedeix o avança un pas. La casella del pas actual es ressaltava en groc i es pintava la peça corresponent.
- **Reproduir / Pausar:** inicia o atura una reproducció animada automàtica.
- **Lliscador de velocitat:** controla el retard entre passos de l'animació (50–2000 ms).

3. Disseny i Implementació

3.1. Patró MVC i Comunicació per Events

El patró MVC separa l'aplicació en tres capes:

- 1) **Model:** encapsula el tauler, el tipus de peces, l'algorisme de cerca i les mètriques d'execució.
- 2) **Vista:** presenta el tauler gràficament, permet configurar la mida i les posicions inicials, i ofereix controls de navegació i reproducció.
- 3) **Controlador:** rep les ordres de la vista, gestiona el cicle de vida del fil de cerca (*SwingWorker*) i actualitza la vista amb els resultats.

La comunicació entre capes es fa exclusivament a través d'events i callbacks.

La cerca de backtracking pot trigar des de mil·lisegons fins a hores o inclús dies per a taulers grans. Executar-la al fil EDT (*Event Dispatch Thread*) bloquejaria la interfície, impedit qualsevol interacció de l'usuari durant la cerca. La solució estàndard és *SwingWorker*, una classe abstracta que encapsula el patró de treball en segon pla: el mètode `doInBackground()` s'executa en un fil separat del, mentre que `done()` s'invoca automàticament al fil

EDT quan la tasca finalitza. El progrés intermedi s'envia al fil EDT amb `SwingUtilities.invokeLater()`, permetent actualitzar les estadístiques en temps real sense bloquejar ni la cerca ni la interfície.

3.2. Capa del Model

La capa del model consta de cinc elements amb responsabilitats clares.

3.2.1. MovementType. Enumeració amb dos valors: `STATIC` i `CONTINUOUS`. Controla el tipus de moviment d'una peça: les peces estàtiques apliquen cada moviment una sola vegada, mentre que les contínues el repeteixen fins arribar al límit del tauler.

3.2.2. PieceType (Enum). Defineix els sis tipus de peces disponibles. Cada constant porta:

- Nom en català i abreviatura.
- `MovementType`: `STATIC` (un sol salt) o `CONTINUOUS` (lliscament).
- Matriu de vectors de moviment `int[][]`. Per a peces estàtiques, cada fila és el desplaçament (*dr, dc*) directe. Per a peces contínues, cada fila és una direcció que es repeteix fins a sortir del tauler o topar una casella visitada.
- `getImagePath(boolean isWhite)`: ruta al recurs de la icona PNG de la peça blanca si `isWhite=true` o negra si `isWhite=false`.

La Taula 1 resumeix els paràmetres i la Figura 1 mostra els vectors de moviment i indica la quantitat de cel·les accessibles *k*, per a cada una de les peces implementades.

Taula 1. PECES IMPLEMENTADES AMB EL SEU TIPUS DE MOVIMENT I NOMBRE DE VECTORS.

Peça	Tipus mov.	# mov.	Abrev.
Cavall	Estàtic	8	C
Reina	Continu	8	Q
Torre	Continu	4	T
Alfil	Continu	4	A
Mussol	Estàtic	8	M
Serp	Estàtic	8	S

Les peces especials *Mussol* i *Serp* s'han definit específicament per a aquesta pràctica. El Mussol es mou amb una variant ampliada del moviment del cavall: salta en forma de L amb passos de magnitud 3 i 1 (vectors $(\pm 3, \pm 1)$ i $(\pm 1, \pm 3)$), en comptes dels passos 2-1 del cavall estàndard. La Serp combina dos tipus de salt: un pas en qualsevol dels quatre sentits ortogonals (vectors $(\pm 1, 0)$ i $(0, \pm 1)$) i dos passos en qualsevol de les quatre diagonals (vectors $(\pm 2, \pm 2)$), donant un total de 8 vectors de desplaçament. La resta de peces tenen els seus moviments clàssics d'un tauler d'escacs.

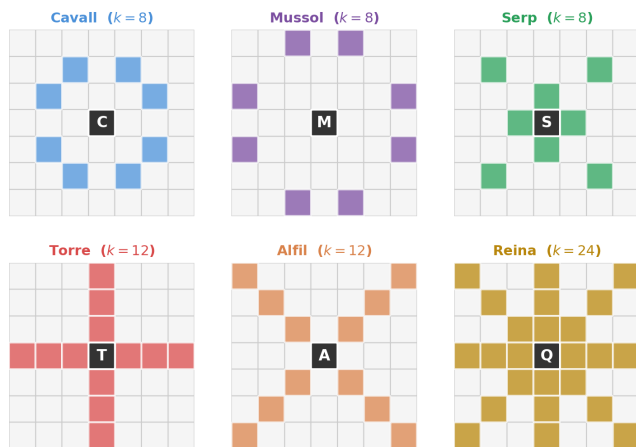


Figura 1. Vectors de moviment de cada peça sobre una graella 7×7 . Les cel·les assolibles des del centre estan marcades en color.

3.2.3. Board. La classe `Board` gestiona l'estat del tauler amb una matriu booleana `boolean[][]` de caselles visitades i un comptador `visitedCount`. Proporciona:

- `getValidMoves(pos, piece)`: retorna les caselles accessibles des de `pos` per `piece` que no han estat visitades. Per a peces contínues, recorr cada direcció cel·la per cel·la fins a sortir del tauler o trobar una casella visitada.
- `getValidMovesSorted(pos, piece)`: igual, però ordena els moviments per l'heurística de Warnsdorff (vegeu Secció 4.2).
- `canCapture(attackPos, attacker, targetPos)`: retorna `true` si `attacker` des de `attackPos` pot assolir `targetPos`. Per a peces estàtiques, comprova si `targetPos` és exactament una aplicació d'algun vector de moviment. Per a peces contínues, recorr cada direcció cel·la per cel·la fins al límit del tauler i comprova si `targetPos` cau en alguna d'aquestes línies.

3.2.4. SolverMetrics. Registra estadístiques de la cerca: nombre d'iteracions, de retrocesos, profunditat màxima assolida i temps transcorregut. Permet monitoritzar l'eficàcia de l'heurística i comparar el comportament per a taulers i peces de diferent dificultat.

3.2.5. Position. Registre immutable que representa una casella del tauler amb les seves coordenades de fila i columna. Es podria substituir un per un simple un array de `int` de dos elements però l'ús de `record` (registre) ens permet referenciar cada coordenada de forma més fàcil i afegir mètodes per a realitzar càlculs sobre la posició. Per exemple: `add(pos)` per a sumar una posició a una altre; útil per a calcular el destí d'un moviment a partir d'una posició inicial.

3.3. Capa de la Vista

3.3.1. MainFrame. `JFrame` principal que compon quatre sub-panells amb `BorderLayout`:

- `ConfigPanel` a l'oest (en un `JScrollPane`).
- `BoardPanel` al centre.
- `ControlPanel` al sud.
- `StatsPanel` al sud (a la dreta de `ControlPanel`).

`MainFrame` és una classe de presentació pura: no manté cap estat de domini ni de navegació. El controlador hi interactua exclusivament a través dels mètodes públics de delegació (`resetBoard`, `updateStats`, `setSolverState`, etc.), que operen amb dades, mai amb components `Swing` directament.

3.3.2. BoardPanel. Panell de dibuix del tauler. Utilitza un `GridLayout` per a distribuir les cel·les del tauler. Cada casella és un `CellPanel` intern que:

- Dibuixa el color de fons (clar/fosc, o verd per a caselles visitades).
- Mostra el número de pas a la cantonada superior esquerra.
- Mostra la icona de la peça a la casella actual de cada peça escalada a la mida de la cel·la.
- Ressalta en groc la casella del pas actual durant la reproducció pas a pas.

El mètode `doLayout` calcula la mida de cel·la màxima entera que cap a l'espai disponible de la següent forma:

$$\text{cellSize} = \min(\lfloor w/\text{cols} \rfloor, \lfloor h/\text{rows} \rfloor)$$

on w és l'amplada del tauler i h l'alçada, i dimensiona la graella a $\text{cols} \times \text{cellSize}$ per $\text{rows} \times \text{cellSize}$, garantint que cada casella sigui un quadrat perfecte fins i tot per a taulers no quadrats ($N \times M$). Les etiquetes de fila i columna s'alineen llegint les coordenades reals de cada `CellPanel` i convertint-les al sistema de coordenades del panell exterior amb `SwingUtilities.convertPoint`. Les caselles responen al clic a través de `mousePressed`, en lloc de `mouseClicked`, per garantir resposta immediata i fiable: `mouseClicked` només s'activa si la posició del ratolí és idèntica entre prémer i alliberar el botó, cosa que falla si l'usuari mou el cursor fins i tot un sol píxel durant el clic [2].

El mètode `updateBoard(path, currentStep, whiteType, blackType)` implementa un renderitzat sense estat: crida primer `clearCells()` per restablir totes les caselles a l'estat buit i després reconstrueix la visualització des de zero recorrent la llista de posicions fins al pas indicat. Per a cada posició visitada s'assigna el número de seqüència en base 1 i es fa un seguiment de la posició més recent de cada peça (paritat parell $\rightarrow$ peça blanca, senar $\rightarrow$ peça negra); al final del recorregut s'hi col·loca la icona corresponent.

El renderitzat de cada `CellPanel` es fa per capes dins de `paintComponent`: primer el fons (blanc o marró per al

tauler, verd clar o verd fosc si ja ha estat visitat, mantenint el patró de colors), després el número de seqüència a la cantonada superior esquerra, a continuació la icona de la peça centrada, i finalment una vora daurada de 3 px si la casella correspon al pas actual.

Quan la cerca finalitza amb solució, `showFullSolution` crida `updateBoard` amb el camí complet i tot seguit elimina tots els ressaltats, de manera que la vista final mostra el tauler completament omplert sense cap casella destacada.

La Figura 2 contrasta el tauler inicial i el tauler amb solució trobada.

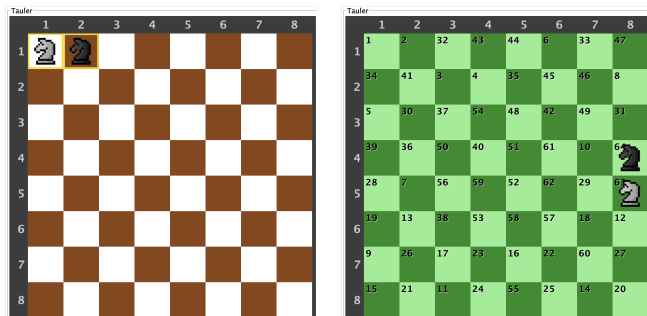


Figura 2. Captura de pantalla del BoardPanel.

3.3.3. ConfigPanel. Panell de configuració que implementa `Scrollable` per garantir que els controls siguin accessibles fins i tot quan la finestra es redimensioni a una alçada menor que l'espai requerit pels controls.

Des del punt de vista tècnic, els *spinners* de posició mostren valors en base 1 ($1 \dots N$ i $1 \dots M$) per a l'usuari, però els mètodes *getter* resten 1 internament per retornar índexs en base 0 ($0 \dots N - 1$ i $0 \dots M - 1$) al controlador. Els botons *toggle* actualitzen el seu color de fons (verd per al bloqueig de mida, taronja per al mode de col·locació) mitjançant `setBackground()` per fer l'estat visible sense modificar el text.

El mètode `validateConfig()` comprova que les posicions estiguin dins dels límits del tauler i siguin distintes, mostrant diàlegs d'error en català si la validació falla. Tota comunicació cap al controlador es fa a través d'un `Runnable` on `onConfigChanged`, registrat exclusivament per `SolverController` en construir l'aplicació. Per tant, el panell no coneix el controlador, només invoca el callback quan detecta canvis de configuració.

La Figura 3 mostra el panell de configuració, a l'esquerra de l'interfície gràfica, on es poden visualitzar totes les opcions disponibles: el botó de bloqueig de tauler

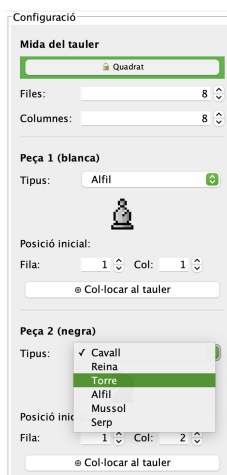


Figura 3. Captura de pantalla del ConfigPanel.

quadrat, la quantitat de files i columnes, el selector de cada una de les dues peces juntament amb la seva posició al tauler.

3.3.4. ControlPanel. Conté sis botons i un lliscador de velocitat organitzats amb `BorderLayout`: els botons al centre i el lliscador a la part inferior. Els botons s'agrupen visualment en dos blocs separats per un `JSeparator` vertical: execució (Iniciar, Aturar, Reiniciar) i navegació (pas anterior, pas següent, reproduir/pausar). Tots els botons utilitzen símbols Unicode (▶, ■, ◯, etc.) combinats amb text en català, i incorporen tooltips descriptius.

El lliscador de velocitat controla el retard de l'animació de reproducció de la solució trobada entre 50 i 2000 ms, amb un valor per defecte de 500 ms. Cal destacar que el lliscador està invertit (`setInverted(true)`): el costat dret correspon a valors baixos de retard (animació ràpida), seguint la convenció intuïtiva on «dreta = més ràpid». En lloc de mostrar els valors numèrics per defecte (50, 2000) als extrems del lliscador, s'utilitza el mètode `setLabelTable()` amb un `Hashtable<Integer, JLabel>` que associa posicions específiques del lliscador amb etiquetes de text personalitzades («Lent», «Ràpid»). Aquest és l'únic mecanisme que l'API de `JSlider` ofereix per personalitzar les etiquetes, dissenyat per permetre col·locar etiquetes a qualsevol posició, no només als extrems.

El panell exposa tres mètodes d'estat explícits (`setIdleState`, `setExecutionState`, `setSolutionState`) que gestionen l'habilitació/deshabilitació dels botons segons la fase actual: durant l'execució de la cerca, tots els botons excepte «Aturar» es deshabiliten per evitar inconsistències; quan hi ha solució disponible, els botons de navegació s'habiliten. El botó de reproducció canvia dinàmicament el seu text entre «▶ Reproduir» i «|| Pausar» segons l'estat intern `isPlaying`.

3.3.5. StatsPanel. Utilitza `GridBagLayout` per organitzar parells etiqueta-valor en una disposició vertical compacta. Cada valor es mostra en negreta per destacar-lo visualment respecte de l'etiqueta. El panell té un `TitledBorder` amb el text «Estadístiques».

Mostra cinc mètriques en temps real: temps transcorregut, nombre d'iteracions, de retrocesos, profunditat màxima i estat actual. El temps es formata automàticament en format MM:SS o HH:MM:SS segons la durada, mitjançant el mètode `formatTime()`. Els nombres grans (iteracions, retrocesos) s'imprimeixen amb separadors de milers locals utilitzant `String.format("%,d", number)` per millorar la llegibilitat.

L'actualització del temps en directe es gestiona amb un `javax.swing.Timer` que s'executa cada 1000 ms (un segon) i recalcula el temps transcorregut des de `timerStartMillis` fins al moment actual. Aquest temporitzador s'inicia quan comença la cerca i s'atura quan finalitza.

Els estats es diferencien amb colors definits com a constants RGB. La Taula 2 mostra els cinc estats possibles amb els seus colors associats.

Taula 2. ESTATS DEL SOLVER I COLORS ASSOCIATS.

Estat	Definició Java
Esperant	Color.GRAY
Cercant	new Color(0, 100, 200)
Solució trobada	new Color(0, 150, 0)
No hi ha solució	new Color(200, 0, 0)
Aturat	new Color(180, 120, 0)

La Figura 4 mostra com ControlPanel i StatsPanel s'han juntat a la part inferior de la interfície gràfica.



Figura 4. Captura de pantalla del ControlPanel i StatsPanel.

3.3.6. PieceIcon. Component reutilitzable que estén JPanel i renderitza la icona d'una peça escalada. Té una mida preferida de 48×48 píxels amb un *padding* intern de 4 píxels. El panell és transparent (`setOpaque(false)`) per integrar-se visualment amb qualsevol fons.

Les imatges es carreguen des del classpath mitjançant `ImageIO.read()` amb un `InputStream`. Per evitar operacions d'I/O repetides, s'utilitza un `HashMap<String, BufferedImage>` estàtic (compartit amb totes les instàncies de `PieceIcon`) com a cache de les imatges indexades per la seva ruta de recurs.

El mètode `paintComponent()` utilitza `Graphics2D` amb dos *rendering hints*: `KEY_ANTIALIASING` perquè les icones es mostren sobre fons de colors variats i les vores irregulars serien especialment visibles, i `KEY_INTERPOLATION_BILINEAR` perquè les imatges s'escalen dinàmicament al tamany de cada casella i la interpolació bilineal evita la pixelació. Com que totes les imatges són quadrades, es calcula la mida màxima del quadrat que cap dins del component (`Math.min(w, h)`) i es centra horitzontalment i verticalment.

Si la imatge no es pot carregar, es dibuixa un *fallback* de text: un cercle amb el nom abreujat de la peça centrat. Les peces blanques utilitzen un fons beix clar (RGB 240, 240, 220) amb text negre, mentre que les peces negres utilitzen un fons marró fosc (RGB 80, 60, 40) amb text blanc. El cercle té un contorn de 1.5 píxels.

3.4. Capa del Controlador

La capa del controlador consta de tres classes: `BacktrackingSolver`, `SolverTask` i `SolverController`. Tot i que

`BacktrackingSolver` implementa l'algorisme de cerca, s'ha classificat al controlador perquè orquestra el procés de resolució (gestiona els torns, el senyal d'aturada i els callbacks de progrés) sense mantenir cap estat de domini propi; l'estat del tauler resideix a `Board`.

3.4.1. BacktrackingSolver. El nucli algorísmic. Abans d'invocar `search()`, el mètode `solve()` reinicia el tauler, la llista de solució i les mètriques; marca les dues caselles inicials com a visitades i les afegeix a la llista; i comprova que les peces no es capturen mútuament a l'inici — si és el cas, retorna `false` immediatament sense iniciar el backtracking. El Listing 1 mostra l'esquelet general de l'algorisme backtracking en pseudocodi.

```

1 boolean search(curPos, curPiece, otherPos,
2   otherPiece) {
3   if (all cells visited) return true;
4   if (stopped) return false;
5
6   for (next : validMovesSorted(curPos, curPiece))
7   {
8     if (canCapture(next, otherPos) || canCapture
9       (otherPos, next))
10      continue;
11
12     visit(next);
13     if (search(otherPos, otherPiece, next,
14       curPiece))
15      return true;
16     unvisit(next); // aquí es produeix el
17       retrocediment real
18   }
19   return false;
20 }

```

Listing 1. Esquelet del backtracking recursiu.

Per a cada candidat `next` es fan dues comprovacions de captura independents: que la peça actual no pugui capturar l'altra des de `next`, i que l'altra peça no pugui capturar la peça actual a `next`. Quan un moviment és vàlid, la posició es marca com a visitada; si la branca no porta a solució, es desfà (`unvisit`). L'intercanvi de rols a la crida recursiva (`otherPos, otherPiece, next, curPiece`) implementa l'alternança de torns. La profunditat màxima és $N \cdot M - 2$.

L'aturada cooperativa es gestiona amb un indicador volatí `boolean stopped`: el bucle comprova `stopped` a cada iteració, i el controlador el posa a `true` quan l'usuari clica *Aturar*.

3.4.2. SolverTask. `SolverTask` és el responsable d'executar i gestionar el cicle de vida de `BacktrackingSolver`: el crea `SolverController`, però és `SolverTask` qui l'executa al fil de fons, registra el callback de progrés i gestiona la seva aturada.

Estén `SwingWorker<Boolean, Void>`. El mètode `doInBackground()` registra un `Consumer<Integer>` al solver com a *callback* de progrés: cada `PROGRESS_UPDATE_INTERVAL` iteracions (1000 per defecte), el solver invoca aquest callback, que

usa `SwingUtilities.invokeLater` per planificar al fil EDT el `Runnable` `onProgress` que el controlador ha registrat en construir la tasca. És doncs el controlador qui defineix l'acció (`view.updateStats()`); `SolverTask` només garanteix que s'executi de manera segura al fil EDT. Aquest disseny desacobla el solver de qualsevol dependència de la vista: `BacktrackingSolver` no coneix ni `Swing` ni la interfície gràfica, només crida un `Consumer` genèric. Un cop registrat el callback, `doInBackground()` executa `solver.solve()`.

El mètode `done()` recupera el resultat i l'envia a un `BiConsumer<Boolean, SolverMetrics>` que el controlador ha registrat en construir la tasca. L'aturada es gestiona amb `stopSolver()`: crida `solver.stop()` per activar l'indicador cooperatiu i `cancel(false)` (mètode de `SwingWorker`) per cancel·lar la tasca sense interrompre el fil, deixant que l'indicador `stopped` aturi la cerca de manera neta.

3.4.3. SolverController. Implementa el rol de controlador en el patró MVC i és l'únic que disposa de l'estat de navegació de la solució: la llista de posicions del camí hamiltonià, el pas actual i els tipus de les dues peces.

Tot el registre d'esdeveniments es concentra al mètode privat `wireListeners()`, invocat al constructor. Aquesta decisió de disseny és intencional: centralitzar el *wiring* al controlador manté `MainFrame` com una classe de presentació pura sense cap coneixement de la lògica de negoci, i evita que la vista hagi de conèixer el controlador directament. El mètode registra els següents listeners de la vista:

- **Iniciar, Aturar, Reiniciar:** botons d'execució de la cerca.
- **Pas anterior, pas següent:** navegació manual per la solució.
- **Reproduir/Pausar:** inicia o atura l'animació automàtica.
- **Lliscador de velocitat:** si l'animació ja està en curs, actualitza el retard del temporitzador en temps real (`animationTimer.setDelay()`) sense aturar ni reiniciar l'animació.
- **onConfigChanged:** callback de `ConfigPanel` que redimensiona el tauler i refresca la previsualització de les peces cada vegada que l'usuari canvia les dimensions o el tipus de peça.

El controlador també és el responsable de l'animació de la reproducció de la solució trobada. Això és una decisió de disseny subjecta a interpretació; no és l'única opció vàlida. La justificació adoptada és que la vista no ha de mantenir cap estat de dades: si el temporitzador residís a la vista, aquesta hauria de tenir accés a la solució (llista de posicions) i a `currentStep` per poder avançar. El controlador en canvi ja gestiona totes dues dades, de manera que el temporitzador encaixa naturalment en la seva lògica.

Quan l'usuari clica *Reproduir*, es crea un nou `javax.swing.Timer` amb el retard actual del llis-

cador; si la reproducció anterior havia arribat al final, el pas es reinicia a 0 automàticament. Cada *tick* crida `stepForward()`: si queden passos, incrementa `currentStep` i actualitza el tauler; si s'ha assolit l'últim pas, atura el temporitzador automàticament. En acabar la cerca, el mètode `onSolvingComplete()` gestiona tres casos: si el solver ha estat aturat, si ha trobat solució (en aquest cas carrega el camí i habilita la navegació) o si no existeix solució; en tots tres casos actualitza l'estat visual de la interfície de manera consistent.

Quan s'inicia la cerca:

- 1) Llegeix els paràmetres de `ConfigPanel` (dimensions, tipus i posicions de les peces blanca i negra).
- 2) Construeix un objecte `Board` i un `BacktrackingSolver`.
- 3) Crea un `SolverTask` i el posa en marxa amb `execute()`.
- 4) El callback de completació (`onSolvingComplete`) rep el resultat al fil EDT, carrega la solució amb `loadSolution()` i actualitza l'estat visual de la vista.

La Fig. 5 mostra el diagrama de classes simplificat amb les relacions entre les tres capes.

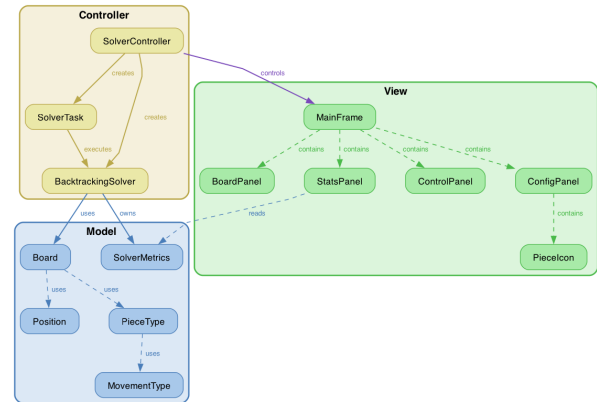


Figura 5. Diagrama de classes i relacions entre les tres capes MVC.

4. Anàlisi de Complexitat

4.1. Complexitat en el pitjor cas

Sigui $S = N \cdot M$ el nombre total de caselles. Les dues peces s'alternen: en cada nivell de l'arbre mou una peça diferent, de manera que els nivells parells corresponen a la peça blanca i els senars a la negra. Siguin k_w i k_b els màxims de moviments vàlids de la peça blanca i negra respectivament. L'arbre té profunditat màxima $S - 2$ (les dues caselles inicials ja estan marcades), amb $(S - 2)/2$ nivells per a cada peça. El nombre total de nodes de l'arbre:

$$V = k_w^{(S-2)/2} k_b^{(S-2)/2} = (k_w k_b)^{(S-2)/2} \quad (1)$$

Quan les dues peces tenen la mateixa quantitat de moviments possibles ($k_w = k_b = k$) això simplifica a k^{S-2} .

La Taula 3 mostra el producte $k_w k_b$ per a totes les combinacions de peces en un tauler $8 \times 8 = 64$, on els valors k son màxims teòrics (suposant que sempre es poden realitzar totes les passes possibles). El nombre total de nodes és $V = (k_w k_b)^{(64-2)/2} = (k_w k_b)^{31}$.

S'utilitza només l'inicial del nom de cada peça per tal de mantenir la taula compacta.

Taula 3. PRODUCTE $k_w k_b$ PER A CADA COMBINACIÓ DE PECES EN UN TAULER 8×8

	C (8)	M (8)	S (8)	A (13)	T (14)	R (27)
C (8)	64	64	64	104	112	216
M (8)	64	64	64	104	112	216
S (8)	64	64	64	104	112	216
A (13)	104	104	104	169	182	351
T (14)	112	112	112	182	196	378
R (27)	216	216	216	351	378	729

Sigui $k = \max(k_w, k_b)$ el nombre total de moviments vàlids (caselles assolibles) i d el nombre de vectors de direcció de la peça ($d \leq 8$). Per a peces estàtiques $k = d$; per a peces contínues $k \gg d$ (p. ex. la Reina té $d = 8$ direccions però $k = 27$ caselles assolibles en un tauler 8×8).

Per a cada node de l'arbre, el mètode `canCapture` s'invoca dues vegades per candidat; en el pitjor cas, recorre les d direccions fins a $\max(N, M)$ passos cadascuna, amb cost $O(d \cdot \max(N, M))$ per crida. Com que $d \leq 8$ és una constant, això és $O(\max(N, M))$. Amb k candidats per node, el cost per node és $O(k \cdot \max(N, M))$.

Juntant-ho tot, tenim un cost computacional:

$$T(S) \in O((k_w k_b)^{(S-2)/2} \cdot k \cdot \max(N, M))$$

Com que el factor polinomial $k \cdot \max(N, M)$ creix molt més lentament que el terme exponencial $(k_w k_b)^{(S-2)/2}$, queda absorbit i la complexitat en el pitjor cas continua sent $T(S) \in O((k_w k_b)^{(S-2)/2})$.

4.2. Heurística de Warnsdorff com a Optimització

L'heurística de Warnsdorff [1] és una regla *greedy* que ordena els moviments candidats creixentment pel nombre de moviments disponibles des de la casella destí cap a caselles no visitades. La intuïció és la següent: si una casella té pocs veïns lliures, és difícil d'arribar-hi més endavant; per tant, convé visitar-la ara, quan encara és accessible. Endarrerint les caselles «populars» —les que es poden assolir des de molts llocs— s'evita quedar-se aïllat en una zona del tauler sense sortida.

L'heurística afegeix un cost addicional per node: el mètode `getValidMovesSorted` es crida una vegada per node i, internament, crida `getValidMoves` per a cada candidat per calcular-ne el grau de Warnsdorff. Cada crida recorre les d direccions fins a $\max(N, M)$ passos, amb cost $O(d \cdot \max(N, M)) = O(\max(N, M))$. Amb k candidats, el

càlcul dels valors heurístics costa $O(k \cdot \max(N, M))$. L'ordenació posterior (`List.sort()`) costa $O(k \log k)$, que queda dominat per $O(k \cdot \max(N, M))$ ja que $\max(N, M) \geq \log k$ per a qualsevol tauler pràctic. El cost total per node és, per tant:

$$O(k \cdot \max(N, M))$$

Combinant amb el nombre de nodes de l'arbre, el cost total per node és el mateix que sense heurística — $O(k \cdot \max(N, M))$ — i queda absorbit per l'exponencial. Per tant, l'heurística no millora el pitjor cas, que continua sent $T(S) \in O((k_w k_b)^{(S-2)/2})$, però sí millora significativament el cas típic. Per tant, en promig és una optimització útil. Per exemple, per a un tauler de 22×22 amb dos cavalls el cost computacional seria el següent:

$$(8 \cdot 8)^{(22 \cdot 22 - 2)/2} = (2^6)^{241} = 2^{6 \cdot 241} = 2^{1446}$$

Si ho volem expressar com a potència de 10:

$$10^e = 2^{1446} \implies e = \log_{10} 2^{1446} = 1446 \log_{10} 2 \approx 435$$

Per tant el cost total teòric seria de 10^{435} , completament intractable. Però amb l'heurística el cost real és molt menor. En una prova del programa amb aquesta configuració s'ha arribat a una solució en menys d'un segon, amb un total de 1109 nodes visitats i només uns 627 *backtracks*.

5. Conclusió i opinió personal

La implementació del recorregut de dues peces alternades, sense capturar-se mútuament, en un tauler d'escacs ha demostrat ser un problema bastant complex amb un cost computacional extremadament elevat per a certes configuracions. Tal com s'ha exposat formalment, aquest cost depèn de la quantitat de moviments que té cada peça i, sobretot, de la mida del tauler. Durant les proves de funcionament del programa ha quedat clar aquest comportament: amb taulells moderadament grans ($> 8 \times 8$) només s'ha pogut verificar si s'arriba a una solució amb peces que tenen pocs moviments (Cavall, Mussol i Serp). Per a la resta de peces és completament inviable esperar a trobar una solució; per a taulells grans, l'aplicació seguiria executant-se indefinidament.

Tot i així, el cost computacional tan elevat ha proporcionat una oportunitat d'optimització de la qual estic molt satisfet d'haver aprofitat. L'heurística de Warnsdorff millora significativament el nombre de passos necessaris per a arribar a una solució en la majoria de configuracions. La prova realitzada amb un tauler 22×22 i dos cavalls (una de les configuracions que minimitza el cost del problema) ho ha posat de manifest molt clarament: el temps seria inassumible sense l'heurística, però pràcticament instantani amb l'heurística activa.

La implementació de la pràctica amb el patró MVC, igual que en la pràctica anterior, ha forçat a pensar de forma crítica quins són els mòduls del programari i quines responsabilitats han de tenir. Ha estat molt útil per a mantenir una bona organització del projecte i permetrà estendre el codi en el futur amb facilitat.

Com és habitual en totes les pràctiques, m'hauria agradat tenir més temps per a continuar millorant certes característiques i afegir funcionalitats. Concretament, hauria estat interessant afegir la funcionalitat de trobar un recorregut per a més de dues peces i també fer estimacions del temps que tardarà l'aplicació en trobar una solució.

Referències

- [1] Conrad_Barski, "Warnsdorff's rule for Knight's tours," Wikipedia, Jan. 2025. [Online]. Available: https://en.wikipedia.org/wiki/Knight%27s_tour#Warnsdorff's_rule:~:text=Warnsdorf%27s%20rule%20is,square%20already%20visited.
- [2] M. Dahmus and F. Kleinschmidt, "mouseClicked vs. mousePressed on Apple vs. Windows," *comp.lang.java.gui*, 2007. [Online]. Available: <https://comp.lang.java.gui.narkive.com/bAHvswQ2/mouseclicked-vs-mousepressed-on-apple-vs-windows>